Day 9 Assignment

```
Intercept = [0.79, 0.561, 0.55, 0.277, -0.057, -0.122, 0.108, 0.103,
0.042, -0.269, -0.367, 0.661, 0.4, -0.78, 0.618, -0.079, -0.645, -
0.484, 0.406, -0.373]

Gradient = [0.745, 0.629, -0.918, -0.567, 0.313, 0.52, 0.72, -0.407,
0.886, -0.202, 0.749, -0.601, 0.381, 0.639, 0.665, 0.859, -0.681, -
0.317, -0.229, 0.063]

Theta_7 = 7
Theta_25 = 25

import math
seismic_response_7 = []
seismic_response_25 = []

if Theta_7 == 7:
    for i in range(len(Intercept)):
        R = Intercept[i] + Gradient[i] * (math.sin(Theta_7) ** 2)
        seismic_response_7.append(round(R, 3))
if Theta_25 == 25:
    for i in range(len(Intercept)):
        R = Intercept[i] + Gradient[i] * (math.sin(Theta_25) ** 2)
        seismic_response_25.append(round(R, 3))

print("Seismic Response for Theta = 7 degrees:", seismic_response_7)
print("Seismic Response for Theta = 25 degrees:", seismic_response_25)
```

```
Seismic Response for Theta = 7 degrees: [1.112, 0.832, 0.154, 0.032,
0.078, 0.102, 0.419, -0.073, 0.424, -0.356, -0.044, 0.402, 0.564, -
0.504, 0.905, 0.292, -0.939, -0.621, 0.307, -0.346]
Seismic Response for Theta = 25 degrees: [0.803, 0.572, 0.534, 0.267,
-0.052, -0.113, 0.121, 0.096, 0.058, -0.273, -0.354, 0.65, 0.407, -
0.769, 0.63, -0.064, -0.657, -0.49, 0.402, -0.372]
```

```
import math
seismic_response_7 = [ round((Intercept[i] + Gradient[i] *
(math.sin(Theta_7) ** 2)), 3) for i in range(len(Intercept)) ]

seismic_response_25 = [ round((Intercept[i] + Gradient[i] *
(math.sin(Theta_25) ** 2)), 3) for i in range(len(Intercept)) ]

print("Seismic Response for Theta = 7 degrees:", seismic_response_7)
print("Seismic Response for Theta = 25 degrees:", seismic_response_25)
```

```
Seismic Response for Theta = 7 degrees: [1.112, 0.832, 0.154, 0.032,
0.078, 0.102, 0.419, -0.073, 0.424, -0.356, -0.044, 0.402, 0.564, -
0.504, 0.905, 0.292, -0.939, -0.621, 0.307, -0.346]
Seismic Response for Theta = 25 degrees: [0.803, 0.572, 0.534, 0.267,
```

```
-0.052, -0.113, 0.121, 0.096, 0.058, -0.273, -0.354, 0.65, 0.407, -
0.769, 0.63, -0.064, -0.657, -0.49, 0.402, -0.372]
```

Double start Opeartor ()**

```
# ** operator (**kwparams)  (**kwargs)
# Returns Dictionary

def well_log_prop(**kwargs):

    return kwargs


well_data = well_log_prop(porosity = 0.22, perm = 200, sw = 0.2,
density = 1.95)
print(well_data)
for key, value in well_data.items():
    print(f" {key} , {value}")
```

```
{'porosity': 0.22, 'perm': 200, 'sw': 0.2, 'density': 1.95}
 porosity , 0.22
 perm , 200
 sw , 0.2
 density , 1.95
```

```
well_data = well_log_prop(0.22, 200, 0.2,1.95)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call
last)
Cell In[8], line 1
----> 1 well_data = well_log_prop(0.22, 200, 0.2,1.95)

TypeError: well_log_prop() takes 0 positional arguments but 4 were
given
```

```
def well_info(well_name, location, **log_properties):
    print(well_name)
    print(location)
    print(log_properties)
    for key, value in log_properties.items():
        print(f"{key}, {value}")

well_info("x-1", "Nile Delta", porosity = 0.18, permeability = 200,
depth = 3000, sw = 0.2)
```

```
x-1
Nile Delta
{'porosity': 0.18, 'permeability': 200, 'depth': 3000, 'sw': 0.2}
porosity, 0.18
```

```
permeability, 200
depth, 3000
sw, 0.2
```

```
well_info("x-1", porosity = 0.18, permeability = 200, depth = 3000, sw
= 0.2)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call
last)
Cell In[11], line 1
----> 1 well_info("x-1", porosity = 0.18, permeability = 200, depth =
3000, sw = 0.2)

TypeError: well_info() missing 1 required positional argument:
'location'
```

```
well_info("x-1", "Nile Delta")
```

```
x-1
Nile Delta
{}
```

```
def well_info(well_name, **log_properties, location):
    print(well_name)
    print(location)
    print(log_properties)
    for key, value in log_properties.items():
        print(f"{key}, {value}")
```

```
  Cell In[13], line 1
    def well_info(well_name, **log_properties, location):
                                               ^
SyntaxError: arguments cannot follow var-keyword argument
```

```
def calculate_vsh(**kwargs):

    GR_log = kwargs.get("GR_log", 0)
    GR_clean = kwargs.get("GR_clean" , 0)
    GR_shale = kwargs.get("GR_shale", 1)

    Vsh = (GR_log - GR_clean) / (GR_shale - GR_clean)
    return round(Vsh, 4)
```

```
calculate_vsh(GR_log = 85, GR_clean = 45, GR_shale = 120)
```

```
0.5333
```

```
Vsh_result = calculate_vsh({
    "GR_log" : 85,
```

```
    "GR_clean" : 45,
    "GR_shale" : 120})
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[17], line 1
----> 1 Vsh_result = calculate_vsh({
      2     "GR_log" : 85,
      3     "GR_clean" : 45,
      4     "GR_shale" : 120})

TypeError: calculate_vsh() takes 0 positional arguments but 1 was given
```

```
def calculate_vsh(**kwargs):

    GR_log = kwargs.get("GR_log", 0)
    GR_clean = kwargs.get("GR_clean" , 0)
    GR_shale = kwargs.get("GR_shale", 1)


    Vsh = (GR_log - GR_clean) / (GR_shale - GR_clean)
    return round(Vsh, 4)

well_log_data = {
    "GR_log" : 85,
    "GR_clean" : 45,
    "GR_shale" : 120}

calculate_vsh(well_log_data)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[21], line 1
----> 1 calculate_vsh(well_log_data)

TypeError: calculate_vsh() takes 0 positional arguments but 1 was given
```

```
calculate_vsh(**well_log_data)
```

```
0.5333
```

```
def calc_SW(Rt, *porosity_values, a=1, m=2, n=2, **kwargs):

    Rw = kwargs.get("Rw", 0.1)
```

```
    print(f"porosity values : {porosity_values}")
    print(f"Archie constants: a =  {a}, m = {m}, n = {n}")
    print(f"Additional information: {kwargs}")

    for phi in porosity_values:
        Sw = ((a *Rw) / (phi **m *Rt))**(1/n)
        print(f"\nfor porosity value {phi:.2f}: water saturation is
{Sw:.3f}\n")

calc_SW(10, 0.15, 0.2, 0.25,a=0.8, m=2.5, n=2, field = "Nile Delta",
Depth = 3000, Rw =0.12 )
```

```
porosity values : (0.15, 0.2, 0.25)
Archie constants: a =  0.8, m = 2.5, n = 2
Additional information: {'field': 'Nile Delta', 'Depth': 3000, 'Rw':
0.12}

for porosity value 0.15: water saturation is 1.050


for porosity value 0.20: water saturation is 0.733


for porosity value 0.25: water saturation is 0.554
```

```
por = [0.15, 0.2, 0.25]

dict = {
    "field" : "Nile Delta", "Depth" : 3000, "Rw" :0.12
}

def calc_SW(Rt, *porosity_values, a=1, m=2, n=2, **kwargs):

    Rw = kwargs.get("Rw", 0.1)

    print(f"porosity values : {porosity_values}")
    print(f"Archie constants: a =  {a}, m = {m}, n = {n}")
    print(f"Additional information: {kwargs}")

    for phi in porosity_values:
        Sw = ((a *Rw) / (phi **m *Rt))**(1/n)
        print(f"\nfor porosity value {phi:.2f}: water saturation is
{Sw:.3f}\n")

calc_SW(10, *por,a=0.8, m=2.5, n=2, **dict )
```

```
porosity values : (0.15, 0.2, 0.25)
Archie constants: a =  0.8, m = 2.5, n = 2
Additional information: {'field': 'Nile Delta', 'Depth': 3000, 'Rw':
0.12}
```

```
for porosity value 0.15: water saturation is 1.050


for porosity value 0.20: water saturation is 0.733


for porosity value 0.25: water saturation is 0.554
```

Lambda Expression

```
#anonymous function

#lambda argument(s) : expression

def AI(Vel, Den):
    Ac_Imp = Vel * Den
    return Ac_Imp

AI(2200, 1.95)

4290.0

AI(1900, 1.7)

3230.0

AcousticImp = lambda velocity, density : velocity * density

print(AcousticImp)

<function <lambda> at 0x0000025AC40EEF20>

AcousticImp(2000, 1.95)

3900.0

vel = [1950, 2000, 2100, 2250, 2300, 2550, 2680, 2750, 2900]

dt = []

def DT(v):
    for i in v:
        dt.append(round(1/i, 6))
    print(dt)

DT(vel)

[0.000513, 0.0005, 0.000476, 0.000444, 0.000435, 0.000392, 0.000373,
0.000364, 0.000345]

vel = [1950, 2000, 2100, 2250, 2300, 2550, 2680, 2750, 2900]
```

```
dt = lambda vels : [round(1/x, 6) for x in vels]

dt(vel)
```

```
[0.000513,
 0.0005,
 0.000476,
 0.000444,
 0.000435,
 0.000392,
 0.000373,
 0.000364,
 0.000345]
```

```
rock_densities = [1.8, 2.5, 3.2, 2.8, 1.9, 3.5]
rock_types = []

def rock_classification(density):
    for value in density:
        if value < 2:
            rock_types.append((value,"Sedimentary"))
        elif 2 < value <= 3:
            rock_types.append((value, "Metamorphic"))
        elif value > 3:
            rock_types.append((value, "Igneous"))

    print(rock_types)

rock_classification(rock_densities)
```

```
[(1.8, 'Sedimentary'), (2.5, 'Metamorphic'), (3.2, 'Igneous'), (2.8,
'Metamorphic'), (1.9, 'Sedimentary'), (3.5, 'Igneous')]
```

```
rock_densities = [1.8, 2.5, 3.2, 2.8, 1.9, 3.5]

rock_types = [(d, "Sedimentary") if d < 2.0 else (d, "Metamorphic") if
d <= 3.0 else (d, "Igneous") for d in rock_densities]

#[expression for item in iterable if condition ]

#[[if condition else condition else condition expression for item in
iterable if condition ]]
print(rock_types)
```

```
[(1.8, 'Sedimentary'), (2.5, 'Metamorphic'), (3.2, 'Igneous'), (2.8,
'Metamorphic'), (1.9, 'Sedimentary'), (3.5, 'Igneous')]
```

```
rock_densities = [1.8, 2.5, 3.2, 2.8, 1.9, 3.5]
rock_densities_2 = [1.7, 2.2, 2.9, 3.5, 1.6, 3.1]
```

```
rock_types = lambda densities: [(d, "Sedimentary") if d < 2.0 else (d,
"Metamorphic") if d <= 3.0 else (d, "Igneous") for d in densities]

rock_types(rock_densities)
```

```
[(1.8, 'Sedimentary'),
 (2.5, 'Metamorphic'),
 (3.2, 'Igneous'),
 (2.8, 'Metamorphic'),
 (1.9, 'Sedimentary'),
 (3.5, 'Igneous')]
```

```
rock_types(rock_densities_2)
```

```
[(1.7, 'Sedimentary'),
 (2.2, 'Metamorphic'),
 (2.9, 'Metamorphic'),
 (3.5, 'Igneous'),
 (1.6, 'Sedimentary'),
 (3.1, 'Igneous')]
```

Map Function

```
#execute function >> iterable
#Alternative for looping


#map(function, iterables)

vel = [1950, 2000, 2100, 2250, 2300, 2550, 2680, 2750, 2900]

dt = []

def DT(v):
    for i in v:
        dt.append(round(1/i, 6))
    print(dt)

DT(vel)
```

```
[0.000513, 0.0005, 0.000476, 0.000444, 0.000435, 0.000392, 0.000373,
0.000364, 0.000345]
```

```
def calc_dt(vel):
    return round(1/vel, 6)

dt  =list(map(calc_dt, vel))
print(dt)
```

```
[0.000513, 0.0005, 0.000476, 0.000444, 0.000435, 0.000392, 0.000373,
0.000364, 0.000345]
```

```
dt  =list(map(lambda v: round(1/v, 6)    , vel))
print(dt)
```

```
[0.000513, 0.0005, 0.000476, 0.000444, 0.000435, 0.000392, 0.000373,
0.000364, 0.000345]
```

Filter Function

```
#filter(function, iterable)

seismic_amplitudes = [30, 55, 42, 67, 90, 25, 51, 75]
high_amplitudes = []

def is_high_amplitude(values):
    for amp in values:
        if amp > 50:
            high_amplitudes.append(amp)
    print(high_amplitudes)

is_high_amplitude(seismic_amplitudes)
```

```
[55, 67, 90, 51, 75]
```

```
seismic_amplitudes = [30, 55, 42, 67, 90, 25, 51, 75]

high_amplitudes = [amp for amp in seismic_amplitudes if amp > 50]

print(high_amplitudes)
```

```
[55, 67, 90, 51, 75]
```

```
seismic_amplitudes = [30, 55, 42, 67, 90, 25, 51, 75]

high_amplitudes = lambda amplitudes : [x for x in amplitudes if x >
50]

print(high_amplitudes(seismic_amplitudes))
```

```
[55, 67, 90, 51, 75]
```

```
seismic_amplitudes = [30, 55, 42, 67, 90, 25, 51, 75]

def high_amp(Amplitude_list):
    return Amplitude_list > 50



high_amplitudes = list(map(high_amp, seismic_amplitudes))

print(high_amplitudes)
```

```
[False, True, False, True, True, False, True, True]
```

```
seismic_amplitudes = [30, 55, 42, 67, 90, 25, 51, 75]

high_amplitudes = list(map(lambda x : x >50, seismic_amplitudes))

print(high_amplitudes)
```

```
[False, True, False, True, True, False, True, True]
```

```
seismic_amplitudes = [30, 55, 42, 67, 90, 25, 51, 75]

def high_amp(Amplitude_list):
    return Amplitude_list > 50


high_amplitudes = list(filter(high_amp, seismic_amplitudes))

print(high_amplitudes)
```

```
[55, 67, 90, 51, 75]
```

```
seismic_amplitudes = [30, 55, 42, 67, 90, 25, 51, 75]

high_amplitudes = list(filter(lambda x : x > 50 , seismic_amplitudes))

print(high_amplitudes)
```

```
[55, 67, 90, 51, 75]
```